ePolicies AI Assistant – Demo Guide

A step-by-step script for demoing the project to Sandeep: what to prepare, what to click, what to say, and how to answer questions. All numbers are from real runs of this project.

1. The project in 30 seconds (say this first)

> "The ePolicies AI Assistant answers questions about our 235 ePolicies documents. You type a question in plain English, and it searches the policies by meaning and by keywords, then Gemini writes an answer using **only** those policies. Every sentence has a source tag you can click to see the exact policy text, the code checks every citation before the answer is shown, and if the policies don't cover something, it says so instead of guessing."

2. Before the demo (15 minutes before)

#	Do this	Check
1	Double-click `start-windows.bat`	Database started, login valid, doctor runs, browser opens
2	If a Google sign-in page opens, sign in with your work account	`[2/5] ... Login is valid`
3	Look at the doctor result	`Connected to PostgreSQL`, `235 documents, 10503 chunks`, Gemini `[ OK ]`; `[WARN] pgvector` is normal
4	Open **http://localhost:3000**	Home page shows "ePolicies AI Assistant"
5	Ask one warm-up question (the COB question below)	An answer with S1, S2… tags appears
6	Click **New chat** so the demo starts clean	Empty chat
7	Close other tabs and notifications; zoom the browser to 110–125%	Easy to read on a shared screen

Backup ready: keep a terminal open in `backend` with the environment active (`..\..\.venv\Scripts\Activate.ps1`), in case the website has a problem (see section 8).

3. Demo flow (about 10–12 minutes)

Time	Show	Key message
0:00	Project summary (section 1)	What it is and why
1:00	Home page	Simple: one question box
1:30	**Knowledge base** page	All 235 policies indexed
2:30	Question 1: COB	Cited, verified answer
5:00	Click a source tag	Answer is traceable to the policy text
6:00	**Diagnostics** switch	How it searched (rounds, sources)

Time	Show	Key message
7:30	Question 2: not covered	It does not make things up
8:30	Question 3: follow-up	It understands the conversation
9:30	How it works (section 5)	Architecture in simple words
11:00	Results, limitations, next steps	Honest status and plan

4. Step-by-step: what to click and what to say

Step 1 – Home page

Do: show http://localhost:3000.

Say: "This is the assistant. There's one question box. Under the title you can see the three ideas behind it: hybrid search, cited answers and verification."

Step 2 – Knowledge base

Do: click **Knowledge base** in the left menu. Scroll the list; open one document.

Say: "These are all 235 ePolicies documents. Each one was split into sections, which I call chunks, 10,503 in total, and each chunk was turned into an embedding, a list of 768 numbers that represents its meaning. That's what lets it search by meaning, not just by exact words."

Step 3 – Question 1 (the main demo)

Do: click **Chat** → **New chat**, then type:

> **What is the policy on coordination of benefits?**

Say while it runs (about 30–40 seconds): "It takes around half a minute because it isn't just one search. You can see the progress steps: it plans the search, searches, checks if it found enough, searches again if needed, writes the answer, then verifies every citation."

When the answer appears, point at:

- The sections in the answer: types of plans we coordinate with and don't, COB benefit credit, order of benefit determination, COB types.
- The **S1, S2… tags** after each sentence.
- The source cards under the answer: real policy names such as *Coordination Of Benefits (COB) - Policy*.

Say: "Every sentence ends with a source tag. In my test run this answer used 8 passages from 2 COB documents, and the code verified all 8 citations before showing it."

Step 4 – Click a source tag

Do: click **S1** (or any tag). The source card shows the exact policy text and its section.

Say: "This is the important part for trust. You never have to believe the AI blindly. Click a tag and you see the exact policy passage the sentence came from."

Step 5 – Diagnostics

Do: turn on the **Diagnostics** switch (top right), ask the question again or open the panel under the answer.

Say: "This is the behind-the-scenes view for developers. It shows the search rounds, why it stopped, the models used, and the final list of passages with their rank in each search: meaning search, keyword search and exact-code search. Those lists are merged with a method called Reciprocal Rank Fusion, so passages that several searches agree on rise to the top."

Then turn Diagnostics off again.

Step 6 – Question 2: something the policies do not cover

Do: ask a question that is **not** in the policies, for example:

> **What is the office parking policy?**

Say: "Now something our policies don't cover. Instead of inventing an answer, it says it couldn't find this in the indexed documents. That's called strict grounding: an answer with no verified source is never shown."

> Tip: if it unexpectedly finds something, that's fine. Click the source to show where it came from.

Step 7 – Question 3: follow-up question

Do: go back to the COB chat (or ask Question 1 again), then type:

> **What about the order of benefit determination?**

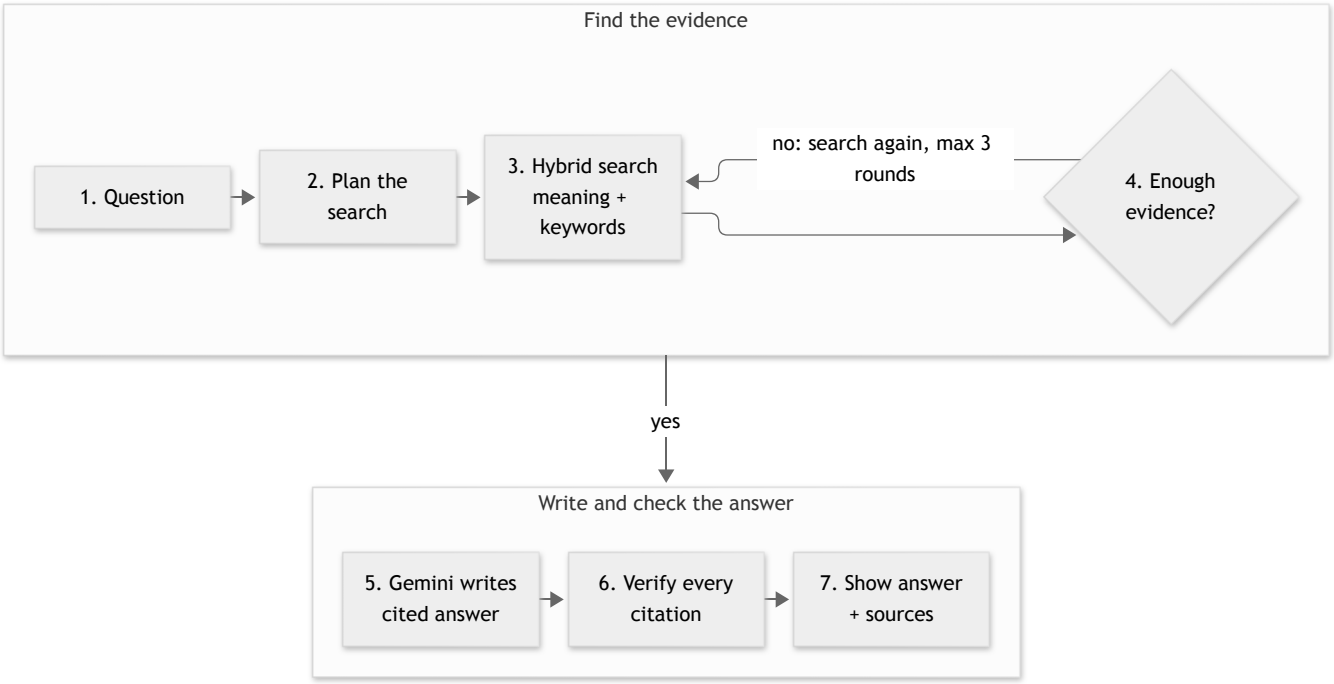
Say: "It remembers the last few turns, so I can ask a short follow-up. It rewrites it into a full question behind the scenes, here about COB, and searches again."

5. How it works (explain in simple words)

Preparing the policies (once, then weekly updates)



Answering a question (every question)



Say it like this:

1. **Preparing the policies (done once, then weekly updates):** the 235 policy files are split into sections. Each section gets an embedding from Google's `gemini-embedding-001` model, and everything is stored in PostgreSQL.
2. **Answering a question:**
   - **Plan:** it finds codes and acronyms in the question and decides if it's simple or complex.
   - **Hybrid search:** three searches at once (by meaning, by keywords, by exact codes), merged into one ranked list.
   - **Evidence check:** Gemini checks "do these passages answer the question?" If not, it searches again, at most 3 rounds.
   - **Answer:** `gemini-2.5-flash` writes the answer using only the found passages, with a source tag on every sentence.
   - **Verify:** code checks that every tag points to a real passage that contains the claim's words and numbers. Weak claims get a second check by Gemini.

Technology in one line each:

Part	Technology	Why
Backend	Python, FastAPI	Fast, simple API; the AI and data tools are in Python
AI	Google Gemini via **Vertex AI** (project `hcb-dev-genai-tools`)	Company-approved access with our Google Cloud login
Embeddings	`gemini-embedding-001`, 768 numbers per chunk	Meaning search
Answers	`gemini-2.5-flash`	Model enabled in our project
Database	PostgreSQL 16 (search + chat history)	One database for everything
Website	Next.js, TypeScript, Tailwind	Modern, fast chat interface

6. Results (real numbers only)

Result	Number
Policy files indexed	**235 / 235**, 0 failed
Searchable chunks	**10,503**
Embeddings created / reused	**9,321** created, **1,182** reused (repeated text embedded only once)
COB demo question	**8 sources**, all **8 citations verified**, 3 search rounds, 5 AI calls, about **39 seconds**
Automated tests	**110** pass on the development machine

Not measured yet (say this honestly): accuracy on a set of real questions, time saved, and cost per question.

7. Recent updates (if he asks "what's new?")

- **Cleaner home page:** removed the example questions, so users ask their own.
- **One-click start and stop:** `start-windows.bat` starts the database, checks the Google login, runs the health check and opens the app. `stop-windows.bat` stops everything.
- **Weekly policy update:** `python scrape_epolicies.py --update` downloads the latest policies, then `python -m app.cli ingest` re-indexes **only the changed files**, so updates are quick and cheap.
- **Fixed** a display warning in the light/dark theme button.
- **pgvector:** not available in the company AppStore. The app uses a built-in fallback that gives the same answers and is fast at our size. pgvector is only needed if we scale up or move to a shared server.

8. If something goes wrong during the demo

Problem	What to do and say
"Could not open a PostgreSQL connection pool"	Database stopped. Run `start-windows.bat` again (it skips what's already running), then click **Retry**.
Gemini / credentials error	Run `gcloud auth application-default login`, restart the backend window, retry.
Website won't load	Show the same thing in the terminal: `python -m app.cli ask "What is the policy on coordination of benefits?" --diagnostics`
Answer is slow	"It runs several steps: planning, up to 3 searches, the answer and verification. Most of the time is the Gemini calls."
An answer looks wrong	Click its source tag: "This is exactly why every sentence is linked to its source, so a person can always check."

9. Limitations (be honest, it builds trust)

- **Runs on one laptop:** not a shared service yet; no user login, so chat history is shared on this machine.
- **Speed:** about 30–40 seconds per answer, mostly Gemini calls.

- **Accuracy not measured yet** on a set of real questions.
- **Very broad questions** ("tell me everything about COB") may be flagged as incomplete; focused questions work best.
- **No pgvector** on the laptop (fallback works fine at our size).
- **Google login expires** now and then; the start script asks you to sign in again.

10. Next steps (what I'd propose)

1. Build a **20-question test set** from real policies and measure accuracy (the evaluation tool already exists: `python -m app.cli evaluate`).
2. **Speed up** answers: fewer search rounds for simple questions, and stream the answer word by word.
3. **Feedback buttons** (👍 / 👎) so users can flag wrong answers.
4. **Shared server** with company sign-on (SSO), a service account for Gemini, and a managed database with pgvector.
5. **Automate the weekly update** (scraper + ingest) on a schedule.

11. Likely questions and short answers

Does it use the internet or general ChatGPT knowledge? No. It only uses our 235 policy files. The only outside connection is Gemini through our company Vertex AI project, which receives the question and the matching passages.

What if the answer isn't in the policies? It says it couldn't find it in the documents and shows no sources. It never shows an answer without a verified source.

How do we know the answer is correct? Every sentence links to the exact policy passage, and the code checks every citation before showing the answer. For important decisions, people should still read the source.

How accurate is it? Not measured on real questions yet. That's my first next step: a 20-question test set with known answers.

How do policies stay up to date? Every week: run the scraper with `--update`, then `ingest`. Only new or changed files are processed.

Why does it take 30–40 seconds? It plans, searches up to 3 times, checks the evidence, writes the answer and verifies it. Most of the time is Gemini. Simple questions are faster.

What is an embedding? A list of 768 numbers that represents the meaning of a piece of text. Texts with similar meanings get similar numbers, so it can find "maximum payment" when you ask about a "reimbursement limit".

What is pgvector, and do we need it? A PostgreSQL add-on that does meaning search inside the database. Our app does the same search itself, with the same answers and fast at 10,503 chunks. pgvector matters only for much bigger data or a shared server.

Is our data safe? The policy files stay on the laptop. Only the question and the matching passages are sent to Gemini in our company Vertex AI project. No API keys are stored in the code; it uses the company `gcloud` login. Data-use approval should be confirmed with compliance.

What does it cost? Not calculated yet. Indexing embedded 9,321 chunks once; each question uses a few Gemini calls (5 for the COB question). Token counts are shown in Diagnostics, so we can estimate a weekly cost.

Can the team use it? Not yet; it runs on my laptop. A shared server with sign-on is the next step.

12. One-page cheat sheet

Need	Command or action
Start everything	Double-click `start-windows.bat`
Stop everything	Double-click `stop-windows.bat`
Website	http://localhost:3000
Health check	`python -m app.cli doctor` (in `backend`, environment active)
Ask in terminal	`python -m app.cli ask "question" --diagnostics`
Google login	`gcloud auth application-default login`
Weekly update	`python scrape_epolicies.py --update`, then in `backend`: `python -m app.cli ingest`
Demo question 1	What is the policy on coordination of benefits?
Demo question 2	What is the office parking policy? (not covered)
Demo question 3	What about the order of benefit determination? (follow-up)